

数字系统设计

Digital System Design

-----微处理器设计1

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信网络工程研究所

Zhejiang University

Spring 2026 – Lec. #12

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室 (可以钉钉、微信或邮件联系) .

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社,2016.
 - 王金明著,《数字系统设计与Verilog HDL》,电子工业出版社,第8版
 - 《数字系统设计-补充讲义》,浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition,电子工业出版社,2005.
 - M.M.Mano, 数字设计(第四版),电子工业出版社,2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**

- 期中考试 **10%**
- Homework Sets **15% (12~20)%**
- Project **5% (5~7)%**
- Class Room Check+小测验等 **10% (7~10)%**
- 作业每周一上交, **截止期为课后一周内**有效。

- **Project**

- Project-1必做 (1 members)
- Project-2可选(成绩加分1%~5%, 但不超过平时成绩范围)

- Finial Exam期末**闭卷**考试 - **60%**

授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

讲义第十章 微处理器设计

-----指令集

以微处理器为中心的 数字系统

Sensor Nets



Cameras



Games



Set-top
boxes



Laptops



Servers



Routers



Robots



Smart
phones



Media
Players



Automobiles



Supercomputers

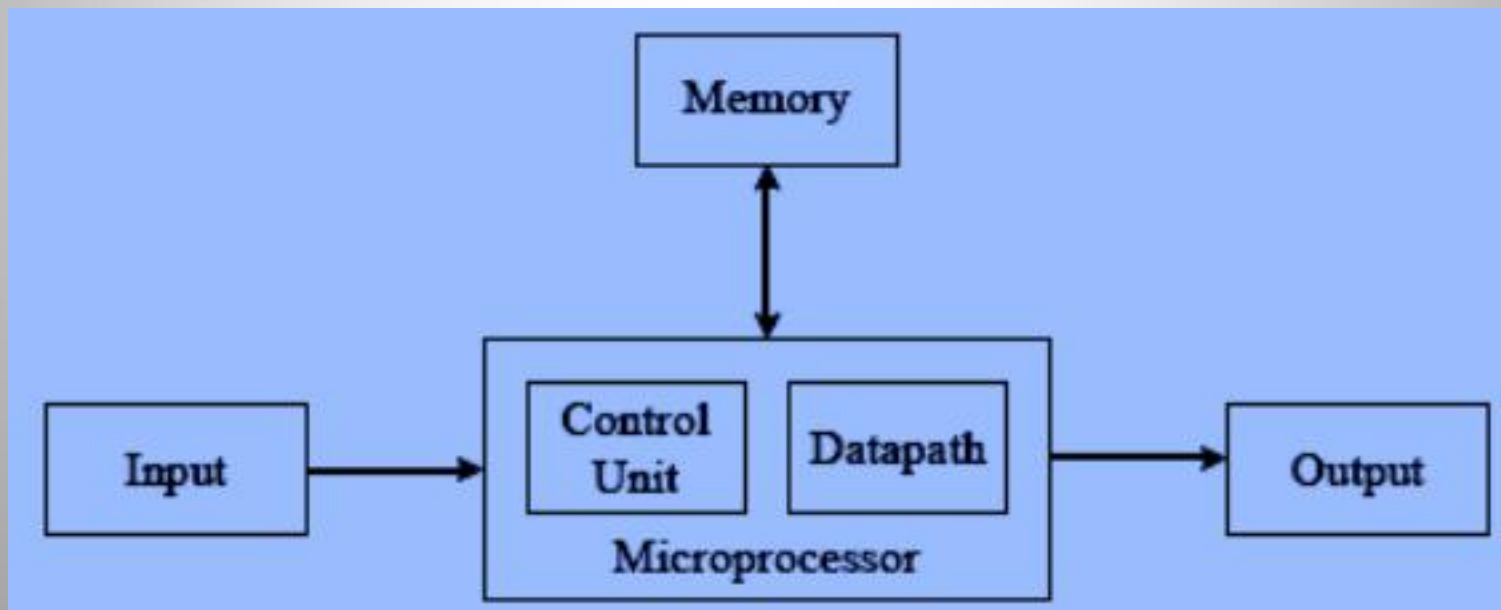


10.1 微处理器结构

- 什么是微处理器？
 - 微处理器由一片或少数几片大规模集成电路组成的中央处理器。
 - 这些电路执行控制部件和算术逻辑部件的功能。
 - 微处理器能完成取指令、执行指令，以及与外界存储器和逻辑部件交换信息等操作，是微型计算机的运算控制部分。
- 微处理器的主要结构
 - 冯诺依曼结构
 - » 将程序存储和数据存储放在同一物理存储空间
 - » 相同的总线
 - » 硬件简单
 - 哈佛结构
 - » 将程序存储和数据存储分别放在不同的物理存储空间
 - » 不同的总线
 - » 灵活、速度快

冯诺依曼计算机模型

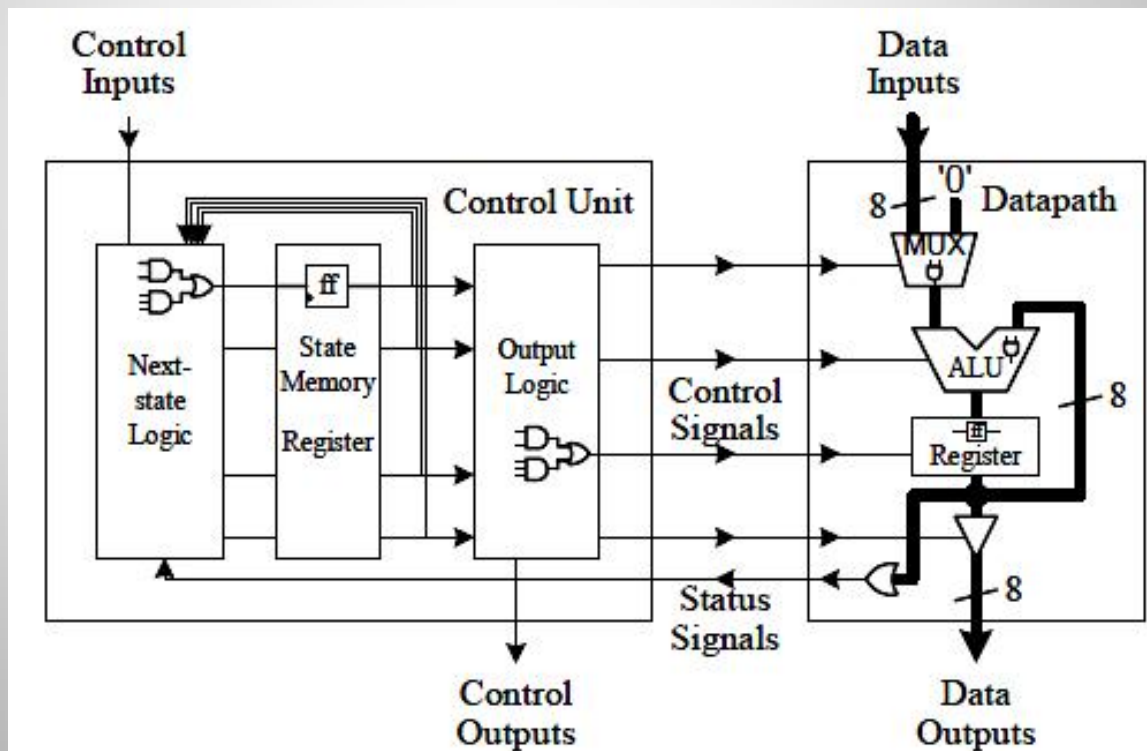
- 四个主要组成部分
 - 输入 INPUT
 - 输出 OUTPUT
 - 存储器 MEMORY
 - 微处理器 CPU



冯诺依曼计算机模型

微处理器内部结构

- 微处理器的两个主要部分：
 - 控制单元
 - » 用于控制数据通路的所有操作，实现微处理器运算的正确性
 - 数据通路
 - » 主要包括运算单元ALU、存储单元(寄存器)及其相互连接



指令集

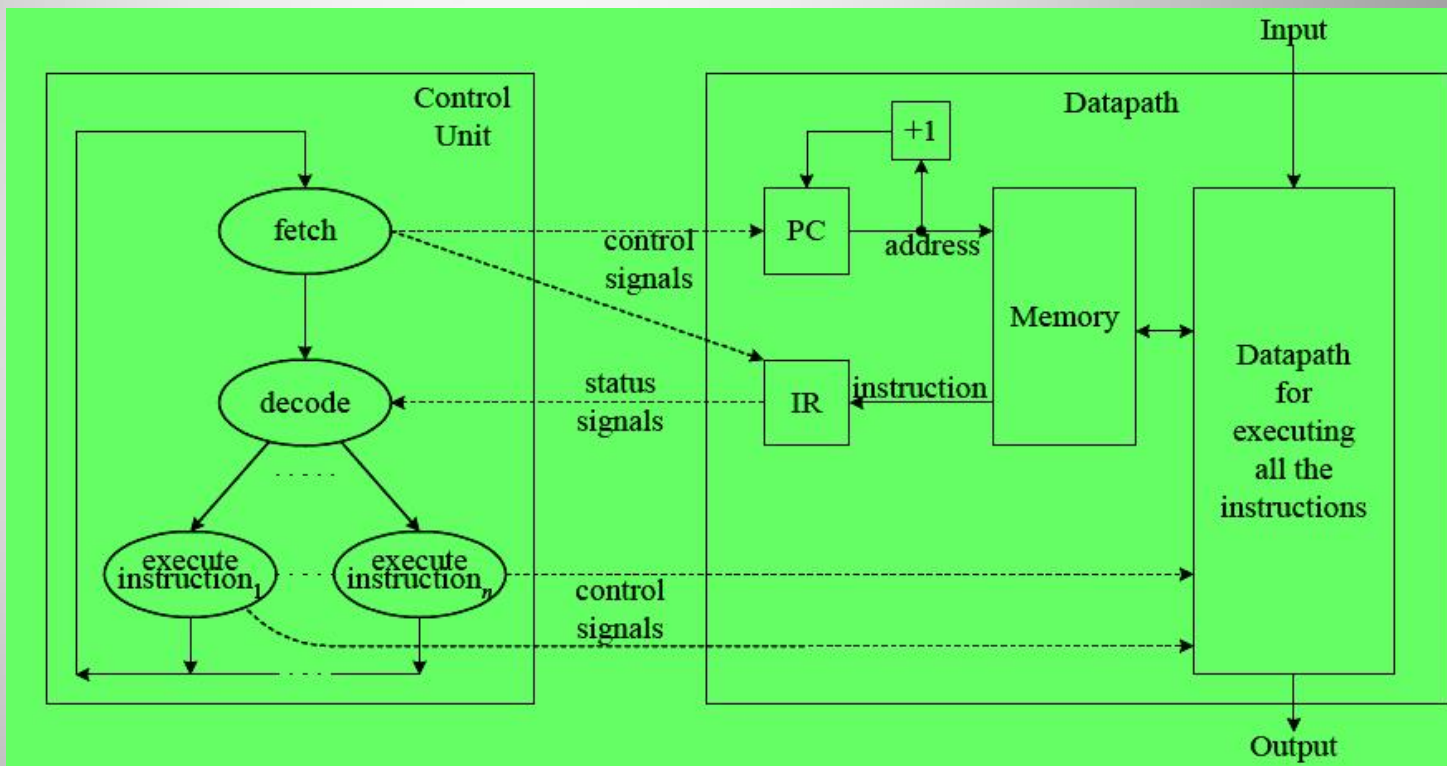
- 专门执行一些指令集的微处理器
- 利用指令集编写不同的程序完成不同的处理任务

需要多少条指令集？

每条指令是什么？

每条指令的操作码
(opcode) 是什么？

指令编码使用多少
位宽的数据？



微处理器整体结构

程序指针 (Program Counter, PC) 寄存器
指令寄存器 (Instruction Register, IR)

RISC
CISC

微处理器的三个执行步骤

- 指令周期

- (1)取指

- » 即从程序存储器中把PC所指向的指令取出, 拷贝到IR;
 - » PC+1指向下一条指令的地址;

- (2)译指

- » 从IR中提取出操作码, 对指令所要做的操作进行翻译;
 - » 决定当前执行那一条指令, 跳转到那一个对应的状态去;

- (3)执行指令

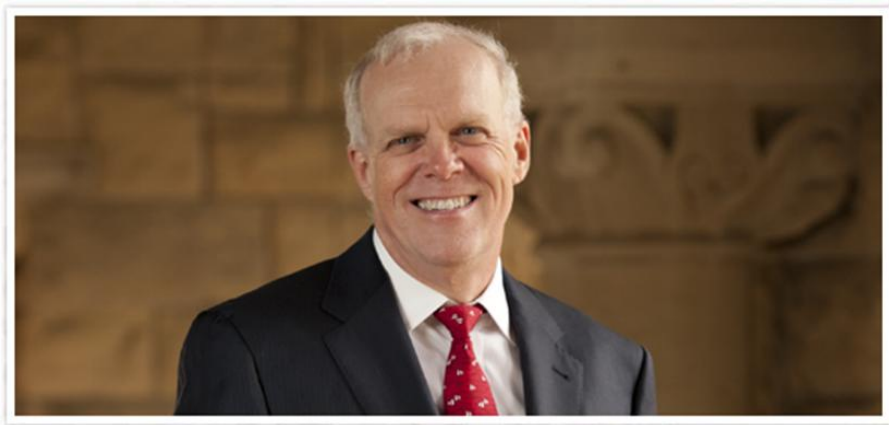
- 每一步将在有限状态机的一个状态被执行。
 - 每个指令通常在一个时钟周期内执行
 - 有些情况下一些存储器操作的指令可能需要两个甚至更长的时钟周期, 这种情况下就需要更多的状态来实现正确的时序。

- 一般而言, 程序的指令都是存储在外部寄存器中的, 所以除了CPU外, 微处理器一般还需要外部存储器以及连接外部存储器与CPU的地址以及数据总线。

10.2 微处理器指令集 IS

- **指令 (Instructions)**
 - 计算机语言里的单词
- **指令集 (Instruction Set)**
 - 计算机的词汇
- **指令需指明要执行的操作和要使用的操作数**
- **机器语言 (*Machine Language*)**
 - 指令编码为二进制数格式
- **汇编语言 (*Assembly Language*)**
 - 符号格式表示各种指令
- **基本指令**
 - 加、减和跳转指令

处理器指令架构ISA



MIPS Founded by J. Hennessy
(the 10th president of Stanford University, 1984)

- MIPS四个原则
 - (1)简洁规整;
 - (2)加快经常性事件的速度;
 - (3)较小的速度更快;
 - (4)好的设计要求良好的妥协。
- MIPS, Intel IA32 (x86), Sun SPARC, PowerPC, IBM 390, Intel IA64, ARM
 - These are all ISAs
- RISC(Reduced Instruction Set Computer)
 - fixed instruction length
 - few instruction formats
 - load/store architecture

设计ARM的原始想法完全来自于上述MIPS研究小组后来发表的论文，两位优秀并敏感的英国工程师Sophie Wilson和Steve Furber看到论文后专门跑到美国去参观实习了一把，回去后说服当时他们的公司老板开始设计ARM1，这个ARM1从1983年10月份项目启动，1年半后流片成功。

汇编语言

- 加法

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code>add a, b, c</code>

- 减法

High-Level Code	MIPS Assembly Code
<code>a = b - c;</code>	<code>sub a, b, c</code>

- 复杂运算

High-Level Code	MIPS Assembly Code
<code>a = b + c - d; // single-line comment</code> <code> /* multiple-line</code> <code> comment */</code>	<code>sub t, c, d # t = c - d</code> <code>add a, b, t # a = b + t</code>

操作数

- 常数 (*Constants*) 和变量
- 寄存器 (*register*)
 - 寄存器组 (register set)
 - 寄存器文件 (register file)
- 寄存器操作

High-Level Code

```
a = b + c;
```

MIPS Assembly Code

```
# $s0 = a, $s1 = b, $s2 = c  
add $s0, $s1, $s2      # a = b + c
```

- MIPS寄存器名称由\$符号开头;
- 变量a、b和c放置在\$s0、\$s1和\$s2;
- 该指令将存在\$s1(b)和\$s2(c)的32位值相加, 32位结果写入\$s0(a)。

MIPS寄存器组

Name	Number	Use
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	procedure return values
\$a0-\$a3	4-7	procedure arguments
\$t0-\$t7	8-15	temporary variables
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	temporary variables
\$k0-\$k1	26-27	operating system (OS) temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	procedure return address

寻址存储器指令

- *load word*

Assembly Code

```
# This assembly code (unlike MIPS) assumes word-addressable memory  
lw $s3, 1($0)    # read memory word 1 into $s3
```

- *store word*

Assembly Code

```
# This assembly code (unlike MIPS) assumes word-addressable memory  
sw  $s7, 5($0)    # write $s7 to memory word 5
```

- MIPS使用字节可寻址存储器；
- 存储器中的每个字节都有一个唯一的地址；

字节寻址组织方式

- ***big-endian 或 little-endian*方式**

- most significant byte, MSB在左侧
- least significant byte, LSB在右侧

- **big-endian机器**

- 字节的编号0从大(most significant)端开始
- IBM 360, Motorola 68K, **MIPS**, SPARC

- **little-endian机器**

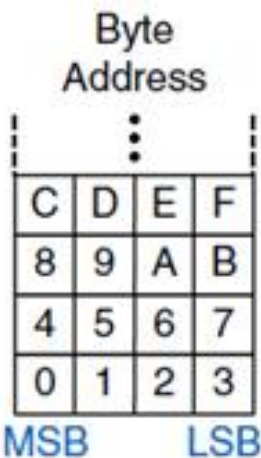
- 字节编号0从小(least significant)端开始
- Intel **x86**, **ARM**, DEC Vax & Alpha,...

- **一个字有四个字节**

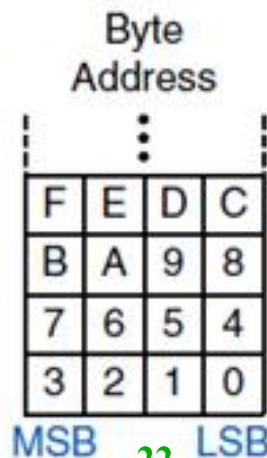
- 字地址相同



Big-Endian



Little-Endian



立即数

- 立即数操作

High-Level Code

```
a = a + 4;  
b = a - 12;
```

MIPS Assembly Code

```
# $s0 = a, $s1 = b  
addi $s0, $s0, 4      # a = a + 4  
addi $s1, $s0, -12    # b = a - 12
```

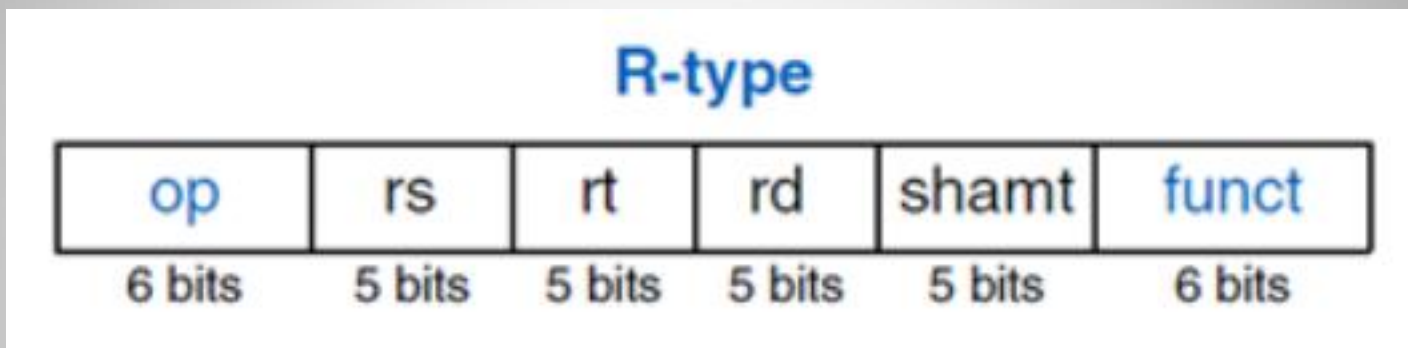
- 立即数是一个16位二进制补码数据;
- 范围是[-32768, 32767]

mips指令格式

- **1) R型-----三个寄存器操作数格式**
 - 用于如`add`和`sub`指令, 有三个寄存器操作数。
- **2) I型-----两个寄存器操作数格式**
 - 用于如`lw`和`sw`指令, 具有两个寄存器操作数和一个16位立即数。
- **3) J型-----无寄存器格式**
 - 有一个26位的立即数, 无寄存器。
- MIPS采用32位指令

机器语言 (machine language)

- R-type Instructions
 - R型是寄存器类型 (*register-type*) 的缩写



- 32位指令
- 六个数字域
 - » *op*, (也称为*opcode*或操作码), 所有的R型指令的*opcode*为0;
 - » *rs*, *rt*, 源寄存器
 - » *rd*, 目标寄存器
 - » *shamt*, 只用于移位操作, 其它R型指令, *shamt*为0。
 - » *funct*, (也称为功能码), 确定特定的R型操作
- 每个字段为五或六比特;

R型指令机器码

- R型指令add和sub的机器码
 - 汇编指令add \$S0, \$S1, \$ S2
 - RS=\$ S1(17), 源数据
 - RT=\$ S2(18), 源数据
 - 结果存到 RD=\$ S0(16)。

Assembly Code

Field Values

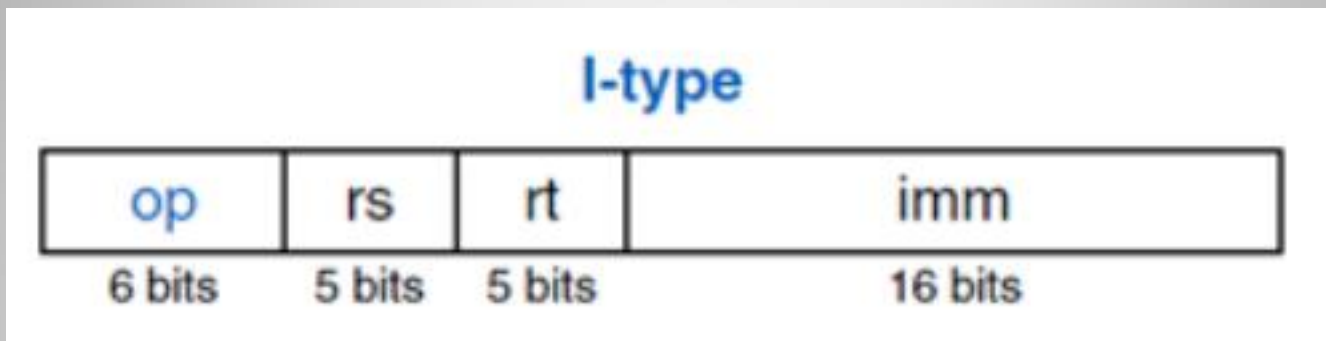
	op	rs	rt	rd	shamt	funct
add \$s0, \$s1, \$s2	0	17	18	16	0	32
sub \$t0, \$t3, \$t5	0	11	13	8	0	34
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	10001	10010	10000	00000	100000	(0x02328020)
000000	01011	01101	01000	00000	100010	(0x016D4022)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

I-Type Instructions

- 名字I型是立即数类型(immediate-type)的缩写



- 四个字段
 - op,
 - rs, 源操作数
 - rt, 如addi和lw用作目标操作数, sw作为另一种源操作数;
 - imm, 源操作立即数;
 - rs和imm始终用作源操作数

I 型指令机器码

Assembly Code	Field Values				Machine Code			
	op	rs	rt	imm	op	rs	rt	imm
addi \$s0, \$s1, 5	8	17	16	5	001000	10001	10000	0000 0000 0000 0101 (0x22300005)
addi \$t0, \$s3, -12	8	19	8	-12	001000	10011	01000	1111 1111 1111 0100 (0x2268FFF4)
lw \$t2, 32(\$0)	35	0	10	32	100011	00000	01010	0000 0000 0010 0000 (0xBC0A0020)
sw \$s1, 4(\$t1)	43	9	17	4	101011	01001	10001	0000 0000 0000 0100 (0xAD310004)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits

I型指令有一个**16位**的立即数字段，
但立即数是在**32位**操作中使用的。

例如，**lw**需要在**32位**的基址寄存器加一个**16位**的偏移量。

符号扩展

如果是正的立即数，**16位**立即数扩展为**32位**，上半部分全为**0**；
如果是负的立即数，扩展为**32位**时上半部分全为**1**。

2的补数符号的扩展不改变它的值。

Signed & Unsigned Number

- If given $b[n-1:0]$ in a register or in memory
- Unsigned value

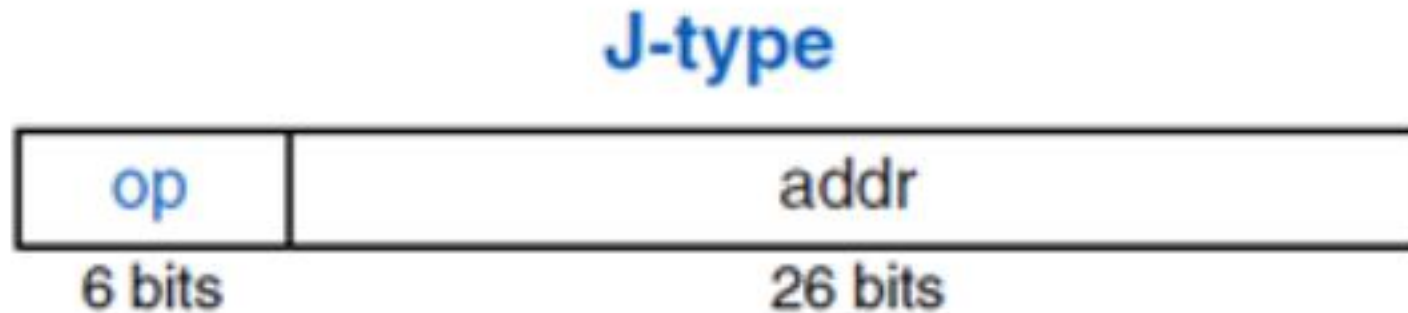
$$value = \sum_{i=0}^{n-1} b_i 2^i$$

- Signed value (2's complement)

$$value = -(b_{n-1} 2^n) + \sum_{i=0}^{n-1} b_i 2^i$$

J-type Instructions

- 名字J型是跳转型 (jump-type) 的缩写
 - 此格式只用跳转指令使用。



- 用单26位的地址操作数, addr
- 用于指定一个地址

例

- 将下面的机器语言代码翻译为汇编语言
- 0x2237FFF1
- 0x02F34022

	Machine Code				Field Values				Assembly Code
	op	rs	rt	imm	op	rs	rt	imm	
(0x2237FFF1)	001000	10001	10111	1111 1111 1111 0001	8	17	23	-15	addi \$s7, \$s1, -15
	2	2	3	7 F F F 1					

	op	rs	rt	rd	shamt	funct	
(0x02F34022)	000000	10111	10011	01000	00000	100010	0 23 19 8 0 34
	0	2	F	3	4	0	2 2

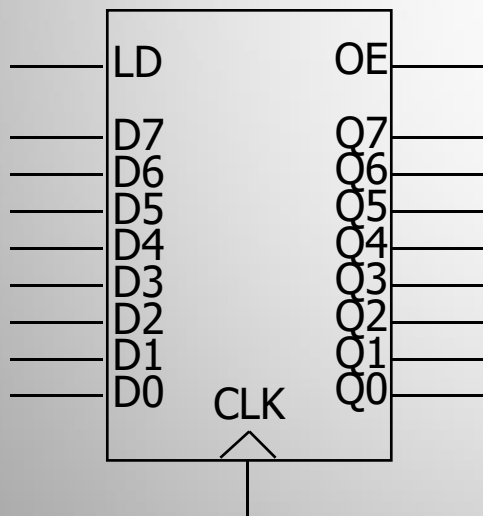
- 把每一条指令写成二进制,
- 在最高六位查找每个指令的opcode, opcode决定如何解释其余位。
- opcode分别为0010002 (810)表示addi和0000002 (010)表示R型指令,
- R型指令的funct字段是1000102(3410), 这表明它是一个sub指令。

讲义第十章 微处理器设计

-----微处理器基础

Registers

- Selectively loaded – EN or LD input
- Output enable – OE input
- Multiple registers – group 4 or 8 in parallel

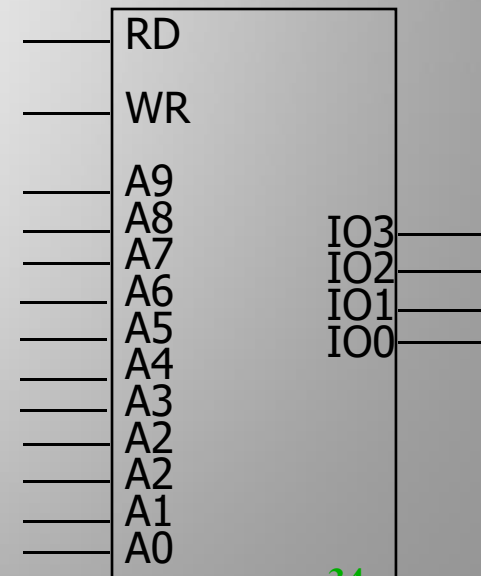


OE asserted causes FF state to be connected to output pins; otherwise they are left unconnected (high impedance)

LD asserted during a lo-to-hi clock transition loads new data into FFs

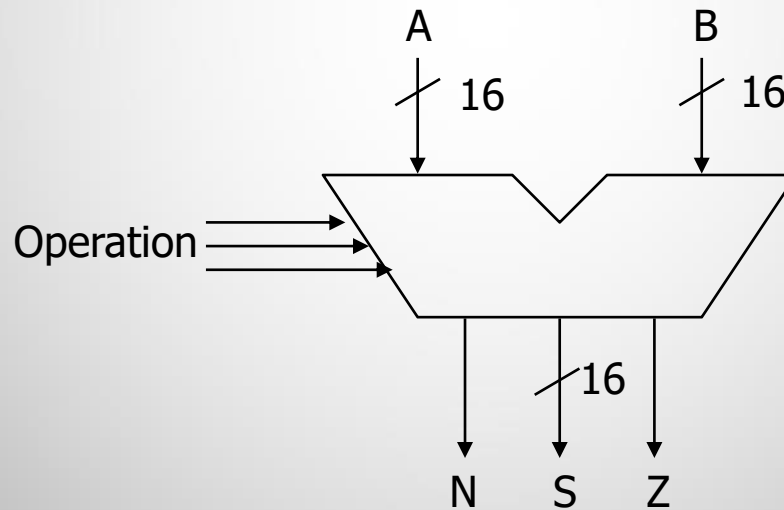
Memories

- Larger Collections of Storage Elements
 - Implemented not as FFs but as much more efficient latches
 - High-density memories use 1-5 switches (transistors) per bit
- Static RAM – 1024 words each 4 bits wide
 - Once written, memory holds forever (not true for denser dynamic RAM)
 - Address lines to select word (10 lines for 1024 words)
 - Read enable
 - » Same as output enable
 - » Often called chip select
 - » Permits connection of many chips into larger array
 - Write enable (same as load enable)
 - Bi-directional data lines
 - » output when reading, input when writing



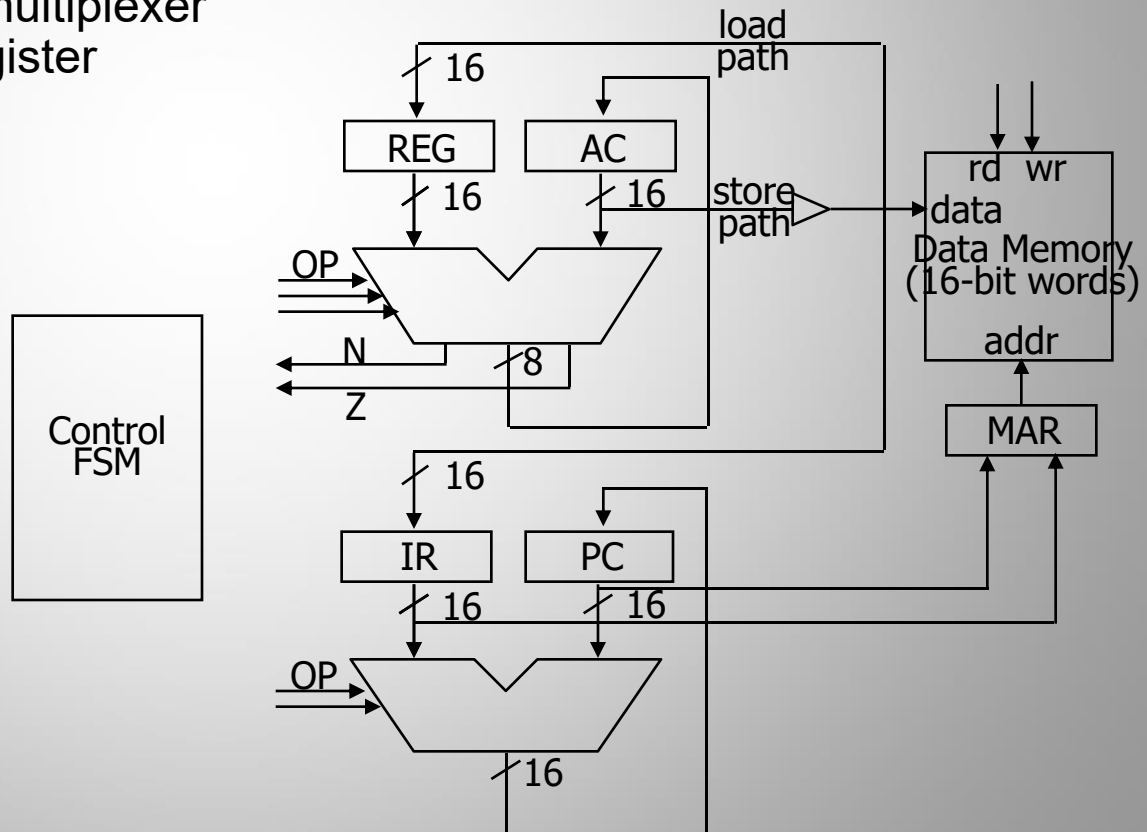
Data Path (ALU)

- ALU Block Diagram
 - Input: data and operation to perform
 - Output: result of operation and status information



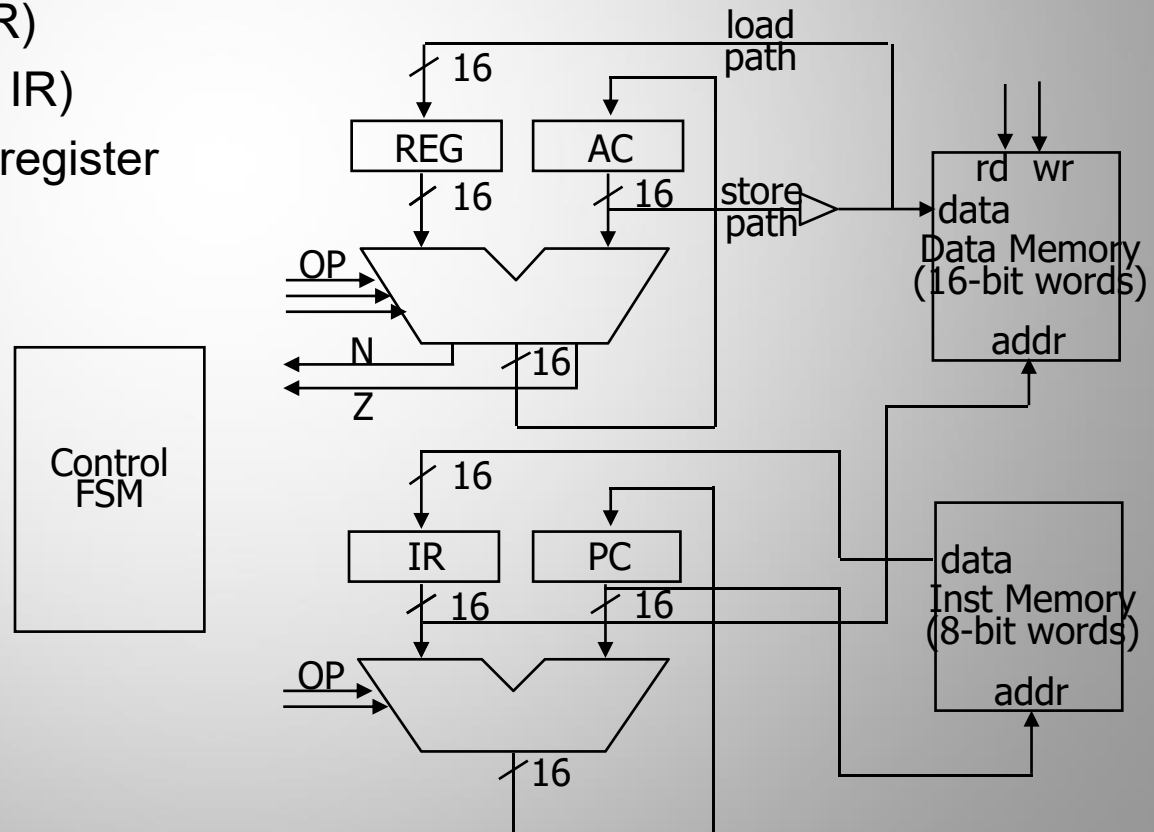
Block Diagram of Processor

- Register Transfer View of Princeton Architecture
 - Which register outputs are connected to which register inputs
 - Arrows represent data-flow, other are control signals from control FSM
 - MAR may be a simple multiplexer rather than separate register
 - MBR is split in two (REG and IR)
 - Load control for each register



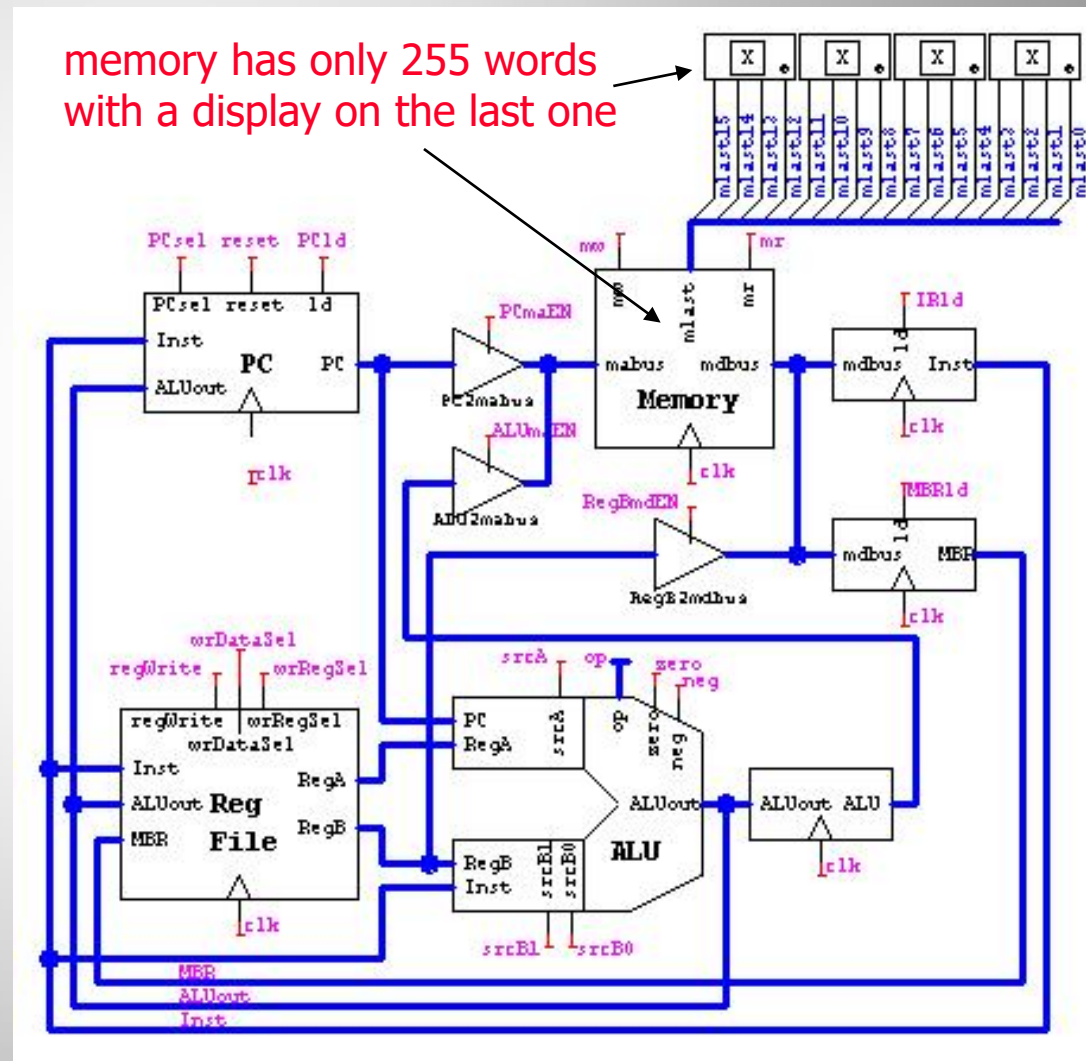
Block Diagram of Processor

- Register transfer view of Harvard architecture
 - Which register outputs are connected to which register inputs
 - Arrows represent data-flow, other are control signals from control FSM
 - Two MARs (PC and IR)
 - Two MBRs (REG and IR)
 - Load control for each register

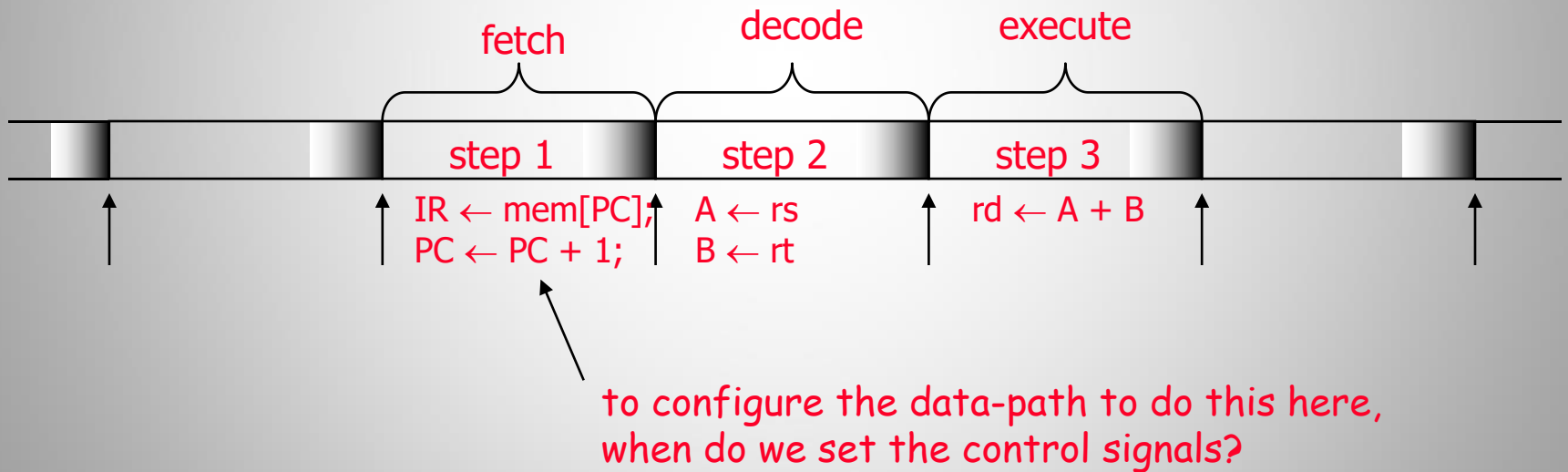


A Simplified Processor Data-path and Memory

- Princeton architecture
- Register file
- Instruction register
- PC incremented through ALU
- Modeled after MIPS rt000 (used in 61C textbook by Patterson & Hennessy)
 - Really a 32 bit machine
 - We'll do a 16 bit version



Review of FSM Timing



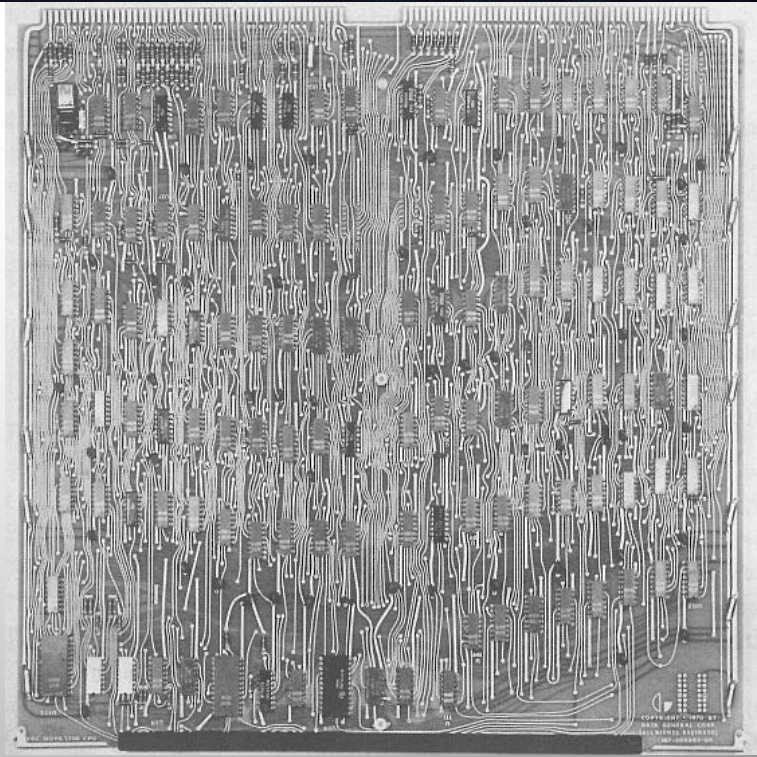
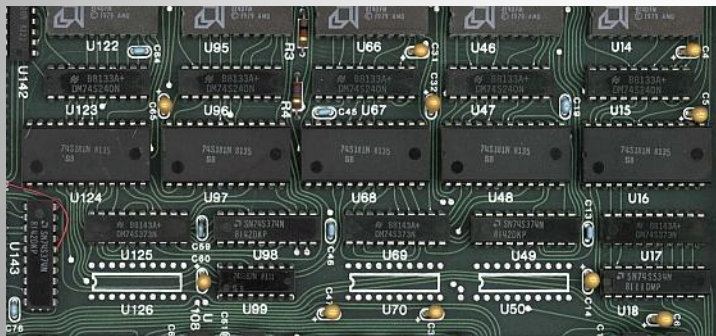
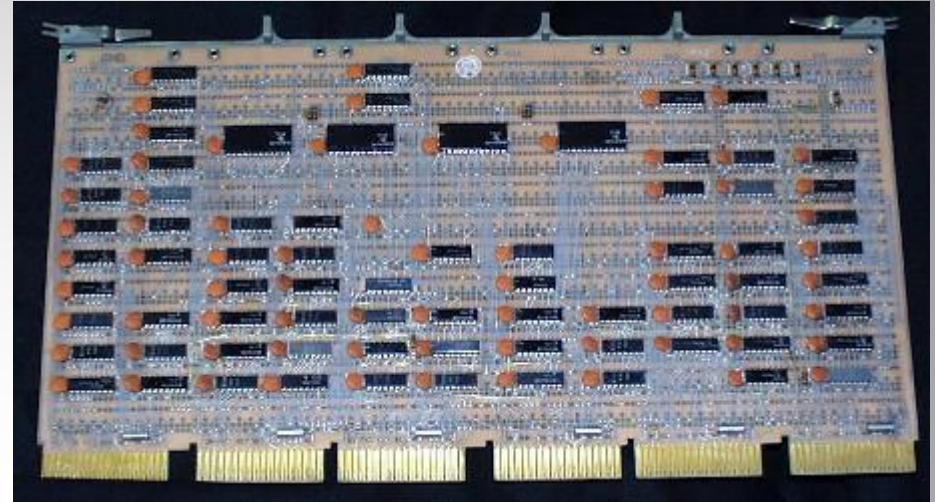
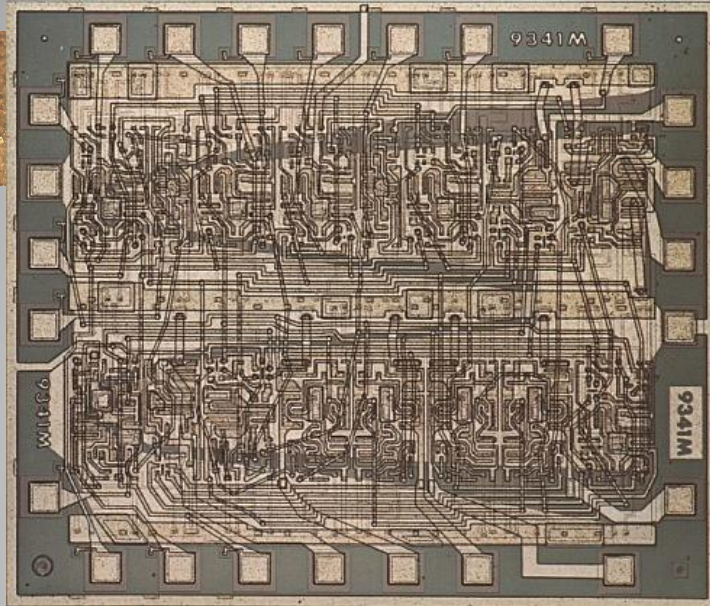
FSM Controller for CPU

- Now we need to repeat this for all the instructions of our processor
 - Fetch and decode states stay the same
 - Different execution states for each instruction
 - » Some may require multiple states if available register transfer paths require sequencing of steps

Alternative Ways to Implement Processor FSMs

- "Random **Logic**" based on Moore and Mealy Design
 - Classical Finite State Machine Design
- Divide and Conquer Approach: **Time-State** Method
 - Partition FSM into multiple communicating FSMs
- Exploit Logic Block Functionality: **Jump** Counters
 - Counters, Multiplexers, Decoders
- Microprogramming: **ROM**-based methods
 - Direct encoding of next states and outputs

74LS181 Computers



课后作业

- **1) 查阅:**

- 国际电路公司的CPU、MCU、DSP:
 - » 8051, PIC, AVR单片机, ARM, MIPS, DSP等微控制器芯片的型号、类型、速度.....
- 图书馆资源: 电子器件天地, 软件
- <ftp://10.13.71.58>

- **2) 作业:**

- 见学在浙大

- **3) 阅读:**

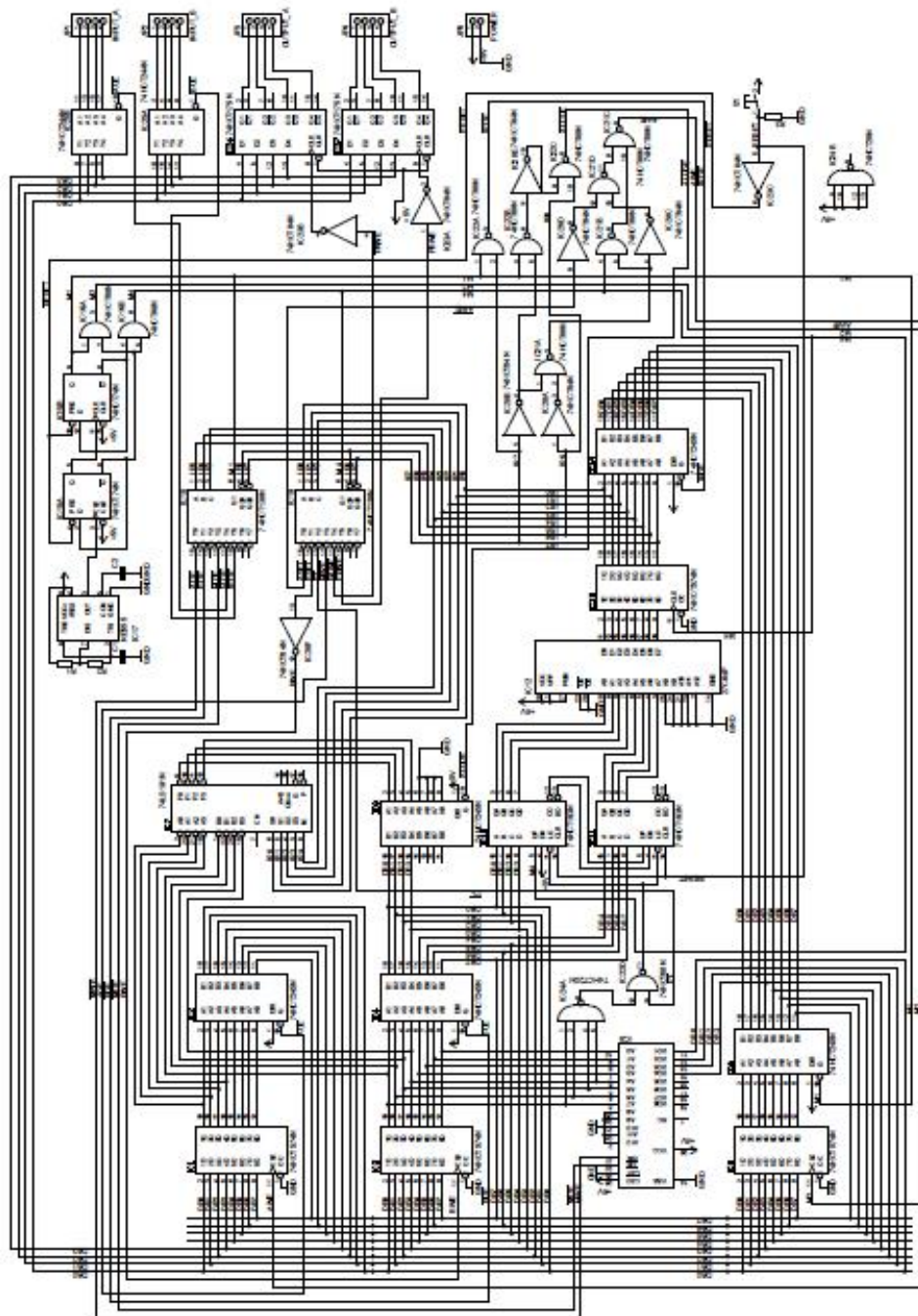
- 复习《数字系统设计与Verilog HDL》
- <http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/>

课后作业

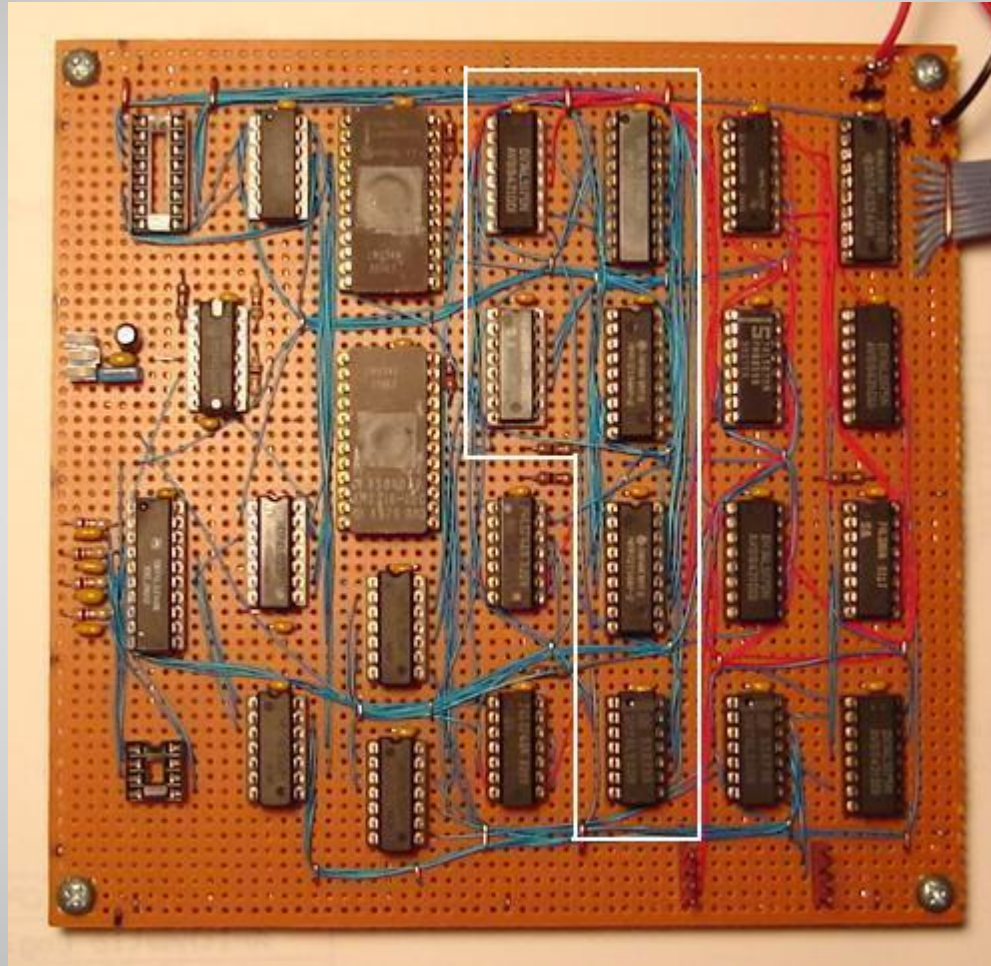
- 4) Project

- Project2（选做平时成绩加分）：用具体逻辑电路描述设计并FPGA实现一个4bit简单CPU实物。（期末考试停课前一周交）

- ALU
- 时钟
- PC计数器
- 寄存器
- 程序存储器
- 寄存器组
- 总线



4bit Simple CPU



参考资料

- www.ttlcpu.com